

PipeLang: DSL Simplificada para Análise de Dados

Bruno Ferreira de Lima 112389

Eng. Computação

Fundação Hermínio Ometto

Araras, Brasil

brunoferreira48@alunos.fho.edu.br

Marina de Souza Pina Oliveira 111838

Eng. Computação

Fundação Hermínio Ometto

Araras, Brasil

marina.oliveira@alunos.fho.edu.br

Resumo—Data analysis by professionals without programming experience—such as biologists, economists, and business analysts—often encounters the complexity of available tools. General-purpose languages, such as Python with Pandas, require technical knowledge that exceeds the competencies of the target audience in these areas. This work presents PipeLang, an interpreted domain-specific language (DSL) designed to allow non-programmer users to describe, execute, and export data analysis pipelines in a declarative and readable way. The language is implemented in Python, using the PLY library for lexical and syntactic analysis, and generates Matplotlib-generated graphs and Pandas-exported CSV files as output. The compiler/interpreter follows the classic phases of a translator: lexical, syntactic, semantic analysis, and interpretation via traversal in the AST. Experimental results with a synthetic dataset of 300 sales records, processed by five representative pipelines, confirm that PipeLang reduces the barrier to entry for data analysis in domains that do not require programming skills.

Index Terms—Domain-Specific Language, Data Pipeline, Compilers, Lexical Analysis, Syntactic Analysis, Python, PLY, Data Visualization

I. INTRODUÇÃO

A crescente disponibilidade de dados em áreas como biologia, economia e ciências sociais gerou uma demanda por ferramentas que permitam a especialistas de domínio realizar análises sem depender de equipes de desenvolvimento de software. Ferramentas de propósito geral, como Python com Pandas e Matplotlib, são poderosas, porém impõem uma curva de aprendizado considerável para profissionais não programadores [5].

Nesse contexto, as Linguagens de Domínio Específico (DSLs) surgem como alternativa promissora. Uma DSL é uma linguagem de programação projetada para um domínio de aplicação específico, com escopo reduzido e vocabulário alinhado ao conhecimento do especialista [1]. Diferentemente das linguagens de propósito geral, uma DSL pode ser aprendida rapidamente por usuários que conhecem profundamente o domínio, mas não têm formação em programação.

Estudos recentes confirmam esse potencial: [2] demonstra que usuários sem experiência profissional em programação conseguem compreender pipelines descritos em uma DSL com maior *correctude* do que quando comparados à leitura de scripts Python equivalentes. Da mesma forma, [4] evidencia que DSLs estaticamente tipadas para ciência de dados são capazes de capturar classes de erros que ferramentas convencionais não detectam.

A motivação central deste trabalho é preencher essa lacuna por meio da criação da **PipeLang**, uma DSL interpretada que permite ao usuário descrever pipelines de análise de dados em sintaxe declarativa e próxima da linguagem natural. Com a PipeLang, um economista pode carregar uma planilha, filtrar registros por região, agrupar por categoria e gerar um gráfico de barras — tudo sem escrever uma linha de Python.

II. FUNDAMENTAÇÃO TEÓRICA

A. Linguagens de Domínio Específico

Uma DSL é uma linguagem de programação especializada em um domínio de aplicação específico, em contraste com linguagens de propósito geral como Python ou Java [1]. Ao focar em um único domínio, uma DSL pode reutilizar o vocabulário e os conceitos daquele contexto, tornando-se mais fácil de aprender e mais eficiente para especialistas da área. A literatura destaca dois tipos principais: DSLs *externas*, com sintaxe própria e parser dedicado, e DSLs *internas* (eDSLs), implementadas como bibliotecas sobre uma linguagem hospedeira.

A PipeLang é uma DSL externa, com gramática, lexer e parser próprios, o que permite maior controle sobre a sintaxe e melhores mensagens de erro para o usuário final.

B. DSLs para Pipelines de Dados

O domínio de pipelines de dados é especialmente adequado para uma DSL. [3] apresenta a SWEL (*Scientific Workflow Execution Language*), uma linguagem de modelagem para workflows intensivos em dados, seguindo os princípios de engenharia orientada a modelos. A SWEL demonstra que uma linguagem independente de plataforma, de alto nível e voltada ao domínio facilita a colaboração entre cientistas de dados e especialistas sem formação técnica.

Em direção semelhante, [4] propõe a Safe-DS, uma DSL estaticamente tipada para ciência de dados capaz de integrar bibliotecas Python de forma segura. Ambos os trabalhos convergem para a mesma conclusão: abstrações de domínio reduzem erros e aumentam a produtividade de usuários não especializados.

C. Gramáticas Formais e Compiladores

A estrutura de um compilador clássico compreende as fases de análise léxica, sintática, semântica e geração de código (ou interpretação). A análise léxica identifica *tokens* por meio de

autômatos finitos determinísticos (AFDs), enquanto a análise sintática valida a estrutura gramatical da entrada utilizando autômatos de pilha e gramáticas livres de contexto (GLCs). A fase semântica verifica coerência de tipos e escopos, e a fase de geração/interpretação produz o resultado final [5].

Para a PipeLang, optou-se por um interpretador baseado em caminhamento de Árvore Sintática Abstrata (AST), em que cada nó da árvore corresponde a uma operação do pipeline e é executado diretamente em Python via Pandas e Matplotlib.

III. DESENVOLVIMENTO E METODOLOGIA

A. Gramática Formal da PipeLang

A gramática da PipeLang foi especificada no formato EBNF (*Extended Backus-Naur Form*). A linguagem adota uma sintaxe declarativa, estruturada em blocos delimitados pelas palavras reservadas `pipeline` e `end`, dentro dos quais o usuário define as etapas do processamento em ordem sequencial.

```

1 programa ::= 'pipeline' STRING bloco 'end'
2 bloco   ::= instrucao+
3 instrucao ::= load | filter | group_by
4           | plot | export
5 load    ::= 'load' STRING
6 filter  ::= 'filter' 'coluna=' STRING
7           'valor=' STRING
8 group_by ::= 'group_by' 'coluna=' STRING
9           'agregacao=' AGREGFUNC
10 plot    ::= 'plot' 'tipo=' PLOTTYPE
11           'eixo_x=' STRING 'eixo_y=' STRING
12 export  ::= 'export' STRING
13 AGREGFUNC ::= 'soma' | 'media' | 'contagem'
14 PLOTTYPE  ::= 'bar' | 'line' | 'scatter'
15 STRING   ::= '"' [a-zA-Z0-9_./-]+ '"'

```

Listing 1. Gramática EBNF da PipeLang

O vocabulário reduzido e a ausência de operadores simbólicos complexos foram escolhas deliberadas para aproximar a linguagem da leitura em linguagem natural, tornando-a acessível ao público-alvo.

B. Ferramentas Utilizadas

O ecossistema de desenvolvimento foi definido integralmente em Python, conforme detalhado na Tabela I.

Tabela I
FERRAMENTAS UTILIZADAS NO DESENVOLVIMENTO DA PIPELANG

Fase	Ferramenta
Lexer / Parser	PLY (Python Lex-Yacc)
AST	Classes Python puras
Manipulação	Pandas [5]
Visualização	Matplotlib [6]
Repositório	GitHub

A PLY (*Python Lex-Yacc*) é uma implementação pura em Python das ferramentas clássicas Lex e Yacc, utilizando o algoritmo de análise LALR(1). Seu uso em contexto educacional é amplamente documentado, sendo considerada adequada para o ensino de compiladores e para o desenvolvimento de linguagens de pequeno porte [5].

O Pandas é a biblioteca de referência para manipulação de dados tabulares em Python, provendo estruturas de dados

como o `DataFrame` e operações de alto nível como filtragem, agrupamento e exportação [5]. A Matplotlib, por sua vez, é a principal biblioteca de visualização científica em Python, com quase duas décadas de desenvolvimento contínuo e utilização por pesquisadores em diversas áreas do conhecimento [6].

C. Arquitetura do Compilador/Interpretador

A PipeLang segue a arquitetura clássica de *front-end* e *back-end* de compiladores. O fluxo de processamento é ilustrado na Figura 1 e descrito a seguir.

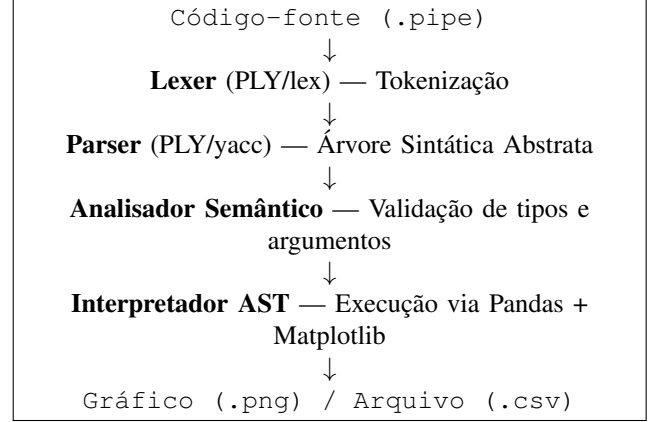


Figura 1. Arquitetura do interpretador da PipeLang

1) *Análise Léxica*: O módulo léxico, implementado com `ply.lex`, define os *tokens* da linguagem por meio de expressões regulares. Os tipos de *tokens* reconhecidos incluem palavras reservadas (`pipeline`, `end`, `load`, `filter`, `group_by`, `plot`, `export`), literais de texto entre aspas duplas, identificadores de atributos (e.g., `coluna=`, `valor=`), e palavras-chave de funções e tipos de gráfico.

2) *Análise Sintática*: O parser, implementado com `ply.yacc`, aplica o algoritmo LALR(1) às regras gramaticais definidas na Seção III-A. Ao reconhecer cada produção, o parser instancia nós da AST correspondentes às operações (e.g., `LoadNode`, `FilterNode`, `PlotNode`). A árvore resultante representa a sequência completa de operações do pipeline.

3) *Análise Semântica*: O analisador semântico percorre a AST antes da interpretação e verifica: (i) a existência de um nó `load` antes de qualquer operação de transformação ou visualização; (ii) a validade dos tipos de gráfico (`bar`, `line`, `scatter`); (iii) a validade das funções de agregação (`soma`, `media`, `contagem`); e (iv) a consistência dos parâmetros obrigatórios de cada instrução.

4) *Interpretação via AST*: O interpretador percorre a AST em ordem de profundidade (*depth-first*) e executa cada nó como uma operação sobre um `DataFrame` Pandas mantido em memória durante a execução do pipeline. O nó `PlotNode` invoca a Matplotlib para gerar e salvar o gráfico, enquanto o nó `ExportNode` serializa o estado atual do `DataFrame` em um arquivo CSV.

IV. RESULTADOS E DISCUSSÃO

Para avaliar a usabilidade e a corretude do interpretador, foi construído um conjunto de dados sintético representando

vendas de um varejo ao longo do ano de 2024, e foram desenvolvidos cinco pipelines de exemplo que exercitam as cinco instruções da linguagem e as três funções de agregação suportadas.

A. Conjunto de Dados Utilizado

O dataset `dados_vendas.csv` contém 300 registros distribuídos em 11 colunas, conforme descrito na Tabela II. As colunas categóricas garantem múltiplas dimensões para filtragem e agrupamento (região, estado, categoria, produto, vendedor, canal), enquanto as colunas numéricas (vendas, quantidade, desconto, custo unitário) permitem aplicar as três funções de agregação. Os dados foram gerados de forma a manter coerência interna: $vendas = preço_unitário \times quantidade \times (1 - desconto/100)$, e `custo_unitario` sempre inferior ao preço de venda, viabilizando análises de margem.

Tabela II
ESTRUTURA DO DATASET `DADOS_VENDAS.CSV`

Coluna	Tipo	Domínio
data	string	2024-01 a 2024-12
regiao	string	Norte, Sul, Leste, Oeste, Centro
estado	string	16 unidades federativas
categoria	string	5 categorias
produto	string	25 produtos distintos
vendedor	string	8 vendedores
canal	string	Online, Loja_Fisica
vendas	numérico	R\$ (com desconto aplicado)
quantidade	numérico	1 a 30 unidades
desconto	numérico	0% a 30%
custo_unitario	numérico	45%–70% do preço

A distribuição de registros é balanceada entre regiões (entre 55 e 64 registros por região) e categorias (entre 52 e 70 registros por categoria), o que evita vieses nas agregações resultantes.

B. Exemplos de Pipelines em PipeLang

A seguir são apresentados três dos cinco pipelines de exemplo desenvolvidos, suficientes para ilustrar as três funções de agregação (soma, media, contagem) e a flexibilidade da DSL.

1) Pipeline 1 — Vendas Totais por Categoria na Região Norte: O Código 2 demonstra um fluxo completo: carregamento, filtragem por valor categórico, agrupamento com soma e geração de gráfico de barras.

```
1 pipeline "Vendas Norte por Categoria"
2   load "dados_vendas.csv"
3   filter coluna="regiao" valor="Norte"
4   group_by coluna="categoria" agregacao=soma
5   plot tipo=bar eixo_x="categoria"
6     eixo_y="vendas"
7   export "resultado_norte.csv"
8 end
```

Listing 2. Pipeline `vendas_norte.pipe`

A execução desse pipeline produz a Figura 2 e um arquivo CSV contendo cinco linhas, uma por categoria, com a soma total de vendas para a região Norte.

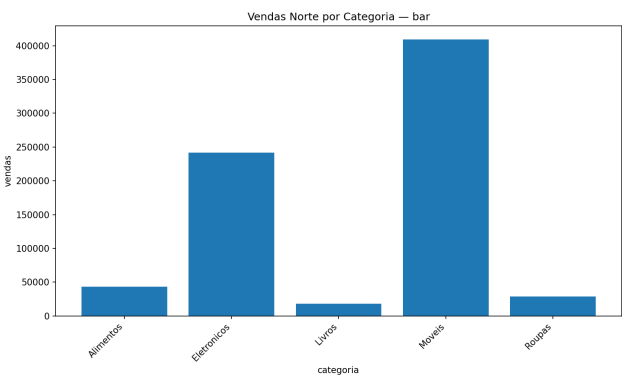


Figura 2. Vendas totais por categoria na região Norte

2) Pipeline 2 — Média de Vendas de Eletrônicos por Região: O Código 3 aplica a função `media` sobre a categoria Eletrônicos, permitindo comparar o ticket médio entre regiões.

```
1 pipeline "Media Eletronicos por Regiao"
2   load "dados_vendas.csv"
3   filter coluna="categoria" valor="Eletronicos"
4   group_by coluna="regiao" agregacao=media
5   plot tipo=bar eixo_x="regiao"
6     eixo_y="vendas"
7   export "resultado_eletronicos.csv"
8 end
```

Listing 3. Pipeline `eletronicos_geral.pipe`

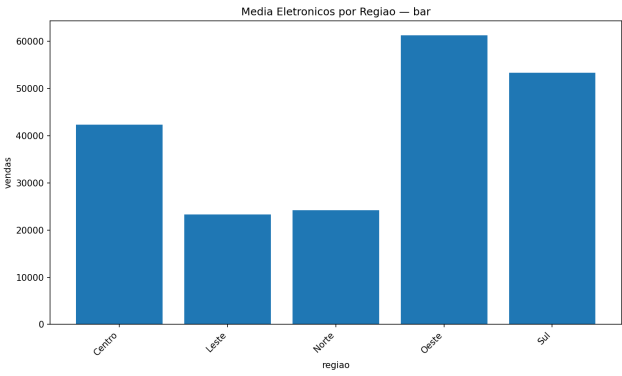


Figura 3. Ticket médio de eletrônicos por região

3) Pipeline 3 — Contagem de Transações de Eletrônicos por Estado: O Código 4 aplica a função `contagem`, útil quando o interesse está no número de ocorrências e não no valor monetário.

```
1 pipeline "Transacoes Eletronicos por Estado"
2   load "dados_vendas.csv"
3   filter coluna="categoria" valor="Eletronicos"
4   group_by coluna="estado" agregacao=contagem
5   plot tipo=bar eixo_x="estado"
6     eixo_y="vendas"
7   export "resultado_estados.csv"
8 end
```

Listing 4. Pipeline `contagem_por_estado.pipe`

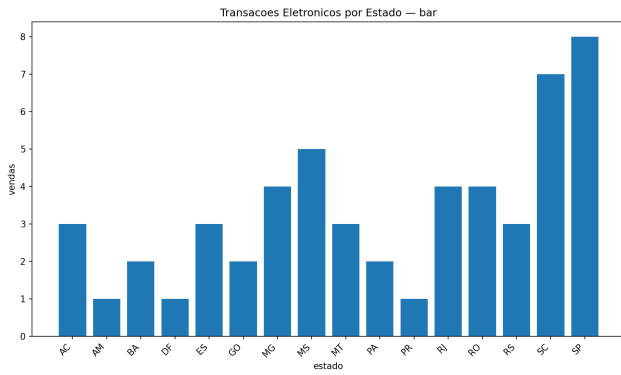


Figura 4. Número de transações de eletrônicos por estado

C. Discussão

Os cinco pipelines desenvolvidos — dos quais três foram aqui apresentados — foram executados com sucesso pelo interpretador, sem ajustes manuais ou intervenção em código Python. Cada pipeline produz dois artefatos: um gráfico em .png e um .csv com o estado final do DataFrame. Comparando o esforço necessário para realizar essas mesmas análises em Python puro, observa-se que cada pipeline em PipeLang é compreensível em uma única leitura, mesmo por usuários sem experiência prévia em programação, ao passo que o equivalente em Pandas exige conhecimento de tipos de dados, expressões booleanas para filtragem e o encadeamento de métodos.

Adicionalmente, foram verificadas as proteções implementadas pelo analisador semântico: pipelines sem instrução `load` inicial, tipos de gráfico não suportados e funções de agregação inválidas são todos rejeitados com mensagens de erro específicas antes da execução. Erros em tempo de execução — como referência a coluna inexistente ou arquivo CSV ausente — também são tratados com mensagens claras, sem expor *stack traces* de bibliotecas internas ao usuário final.

V. CONCLUSÃO

Este trabalho apresentou a PipeLang, uma DSL externa interpretada destinada à descrição declarativa de pipelines de análise de dados por usuários sem formação em programação. O projeto cobriu integralmente as fases clássicas de um compilador — análise léxica, sintática, semântica e interpretação — utilizando a biblioteca PLY para o front-end e as bibliotecas Pandas e Matplotlib para a execução das operações sobre dados tabulares.

Os resultados obtidos a partir de cinco pipelines de exemplo, aplicados sobre um dataset sintético de 300 registros, confirmam que a linguagem é capaz de expressar de forma concisa as operações típicas de análise exploratória — carregamento, filtragem, agrupamento, visualização e exportação — com vocabulário próximo da linguagem natural. A separação clara entre as fases do compilador também trouxe ganhos de manutenibilidade: cada componente pode ser modificado ou estendido independentemente.

Mesmo no escopo atual, a PipeLang demonstra que uma DSL bem projetada pode reduzir significativamente a barreira de entrada para a análise de dados, alinhando-se à literatura recente que aponta os ganhos de produtividade e correteza oferecidos por linguagens específicas de domínio.

REFERÊNCIAS

- [1] P. Heltweg, G.-D. Schwarz, R. Dirk, and Q. Felix, “An Empirical Study on the Effects of Jayvee, a Domain-Specific Language for Data Engineering, on Understanding Data Pipeline Architectures,” *Software: Practice & Experience*, 2025. Disponível em: <https://oss.cs.fau.de/2025/02/20/research-paper-an-empirical-study-on-the-effects-of-jayvee>
- [2] P. Heltweg *et al.*, “Can a domain-specific language improve program structure comprehension of data pipelines? A mixed-methods study,” *Empirical Software Engineering*, Springer, nov. 2025. DOI: <https://doi.org/10.1007/s10664-025-10746-7>
- [3] R. Salado-Cid, A. Vallecillo, K. Munir, and J. R. Romero, “SWEL: A Domain-Specific Language for Modeling Data-Intensive Workflows,” *Business & Information Systems Engineering*, vol. 66, no. 2, pp. 137–160, Springer, 2024. DOI: <https://doi.org/10.1007/s12599-023-00826-7>
- [4] G. Lüttgen *et al.*, “Safe-DS: A Domain Specific Language to Make Data Science Safe,” *arXiv preprint arXiv:2302.14548*, 2023. Disponível em: <https://arxiv.org/abs/2302.14548>
- [5] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conference (SciPy)*, 2010, pp. 51–56. DOI: <https://doi.org/10.25080/Majora-92bf1922-00a>
- [6] S. Kang *et al.*, “Mastering data visualization with Python: practical tips for researchers,” *PubMed Central (PMC)*, 2023. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10728683/>
- [7] Repositorio GitHub do projeto Disponível em: https://github.com/Bruno48Ferreira/PipeLang_DLS